

# Planning Journalier

Sprint de 4 semaines — 20 jours ouvrés



## Toutes les tâches, jour par jour, par rôle :

Auth, CRUD, ingestion MQTT, géofencing, congés, paie, tests, déploiement.



### DURÉE

4 semaines — du lundi 20 juillet au vendredi 14 août 2026

### ÉQUIPE

2 Backend, 1 Frontend Web, 1 Mobile, 1 QA — supervisés par le Lead

### OBJECTIF

Livrer l'ensemble des modules du cahier des charges en 1 mois

### RYTHME

Démo interne chaque vendredi fin de semaine

## Comment lire ce planning

Hypothèse : démarrage le lundi 20 juillet 2026, 5 jours ouvrés par semaine, 4 semaines pleines (20 jours), livraison le vendredi 14 août 2026 — dans le délai d'un mois demandé. Chaque vendredi est une démo interne : c'est le moment de vérité qui évite les mauvaises surprises en fin de mois.

**Équipe** : Backend 1 (Auth, Chantiers, Employés), Backend 2 (Ingestion MQTT, Pointages, Paie), Frontend Web, Mobile, QA. Toi (Lead) supervises, débloques, et intervients techniquement si un stagiaire est absent — répartis alors ses tâches du jour sur les autres profils compatibles.

**Règle simple** : si un jour prend du retard, ne compresse jamais les tests de la semaine 4 (section 8 du cahier des charges). Mieux vaut livrer une fonctionnalité en moins mais fiable, qu'un module paie qui donne de mauvais chiffres.

## Semaine 1 — Fondations : Auth, CRUD, ingestion MQTT

| Jour                                     | Backend 1<br>(Auth / Chantiers / Employés)                                      | Backend 2<br>(MQTT / Pointages / Paie)                                                                   | Frontend Web                                                                              | Mobile                                                                       | QA                                                                                                    |
|------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>J1</b><br>Lun<br>20/07                | Init repo NestJS, structure feature-first, config des variables d'environnement | Docker Compose (PostgreSQL + broker MQTT), schéma Prisma initial (chantiers, employés, pointages)        | Init projet Vite + TailwindCSS, structure feature-first, routing de base                  | Init projet Expo (React Native), structure feature-first, navigation de base | Rédaction des scénarios de test pour les 5 critères d'acceptation du cahier des charges               |
| <b>J2</b><br>Mar<br>21/07                | Module Auth : POST /auth/login, génération JWT, guards NestJS                   | Module Employés : CRUD complet (create/read/update/delete) + DTO                                         | Écran de connexion admin, intégration de l'appel API /auth/login                          | Écran de connexion employé (mobile)                                          | Vérifier que chaque environnement démarre correctement ; tester le login (cas valides/invalides)      |
| <b>J3</b><br>Mer<br>22/07                | Module Chantiers : CRUD complet (nom, position, rayon de tolérance)             | Module Ingestion MQTT : connexion au broker, écoute du topic, log des messages bruts reçus               | CRUD Employés côté dashboard (liste, ajout, édition, suppression)                         | Écran Profil (lecture seule) branché sur l'API employé connecté              | Tester le CRUD Employés (champs obligatoires, doublons de badge, etc.)                                |
| <b>J4</b><br>Jeu<br>23/07                | Service de géofencing (formule de Haversine) + tests unitaires                  | Traitement des messages MQTT reçus → déduction Entrée/Sortie, écriture en base pointages                 | CRUD Chantiers côté dashboard + intégration de la carte Leaflet (affichage des chantiers) | Écran Historique — interface avec données mockées en attendant l'API         | Tester le CRUD Chantiers ; valider le calcul Haversine sur des cas de distance connus                 |
| <b>J5</b><br>Ven<br>24/07<br><b>DÉMO</b> | Endpoint d'association badge RFID ↔ employé                                     | Gestion des pointages asynchrones : rejeu du backlog, tri par timestamp RTC (pas par heure de réception) | Carte avec alertes hors-zone (données mockées), finitions listes employés/chantiers       | Brancher l'écran Historique sur l'API réelle /pointages                      | Test asynchrone (pointage vieux de 24h classé à sa date d'origine) + démo interne de fin de semaine 1 |

## Semaine 2 — Pointages avancés &amp; Congés

| Jour                               | Backend 1<br>(Auth / Chantiers / Employés)                                          | Backend 2<br>(MQTT / Pointages / Paie)                                                     | Frontend Web                                                                      | Mobile                                                                    | QA                                                                             |
|------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| <b>J6</b><br>Lun<br>27/07          | Endpoint GET /pointages avec filtres (employé, période)                             | Alertes temps réel vers le dashboard (WebSocket ou polling) en cas de pointage hors-zone   | Tableau d'historique des pointages + affichage des alertes en temps réel          | Écran Congés (mobile) — formulaire de demande : dates de début/fin, motif | Test de précision GPS (satellites < 3) ; test de déclenchement du géofencing   |
| <b>J7</b><br>Mar<br>28/07          | Module Congés : CRUD des demandes + statuts (en attente / approuvé / rejeté)        | Robustesse de l'ingestion MQTT : gestion des rafales de messages, file d'attente si besoin | Écran Congés admin — boîte de réception, boutons approuver / rejeter              | Brancher le formulaire de congé sur POST /conges                          | Tester le workflow congés de bout en bout (demande mobile → traitement admin)  |
| <b>J8</b><br>Mer<br>29/07          | Endpoint PATCH /pointages : correction manuelle d'un pointage par l'admin           | Table logs_ingestion : traçabilité de chaque message MQTT reçu (debug terrain)             | Fonction de correction d'un pointage dans le dashboard                            | Écran de suivi du statut des demandes de congé (mobile)                   | Tester la correction manuelle d'un pointage ; vérifier la traçabilité des logs |
| <b>J9</b><br>Jeu<br>30/07          | Tests unitaires du module Pointages (Haversine, transitions d'état) + revue de code | Tests unitaires du module Congés + revue de code croisée avec Backend 1                    | Filtres et recherche sur le tableau de pointages (par employé, chantier, période) | Gestion des erreurs réseau et des états de chargement (mobile)            | Test de charge MQTT : rejouer plusieurs centaines de messages en rafale        |
| <b>J10</b><br>Ven<br>31/07<br>DÉMO | Correction des bugs remontés sur les modules Pointages / Chantiers                  | Correction des bugs remontés sur les modules Congés / Ingestion                            | Corrections UI et préparation de la démo interne                                  | Corrections UI et préparation de la démo interne                          | Recette des modules Pointages + Congés ; démo interne de fin de semaine 2      |

## Semaine 3 — Paie, sécurité &amp; finalisation fonctionnelle

| Jour                                      | Backend 1<br>(Auth / Chantiers / Employés)                                  | Backend 2<br>(MQTT / Pointages / Paie)                              | Frontend Web                                                                | Mobile                                                                 | QA                                                                            |
|-------------------------------------------|-----------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------|------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>J11</b><br>Lun<br>03/08                | Module Paie : agrégation timesheet + congés + taux horaire                  | Génération de l'export PDF du rapport de paie (pdf-lib / Puppeteer) | Écran « Générer rapport de paie » (sélection du mois, bouton de génération) | Amélioration UX : cache local, indicateur de statut de synchronisation | Tester le calcul de paie sur des cas simples connus (heures × taux horaire)   |
| <b>J12</b><br>Mar<br>04/08                | Finaliser les règles de calcul (heures supplémentaires, absences, arrondis) | Génération de l'export Excel du rapport de paie (exceljs)           | Prévisualisation et téléchargement du rapport (PDF et Excel)                | Corrections UI, tests sur différentes tailles d'écran                  | Vérifier la cohérence des exports PDF/Excel avec les données de la base       |
| <b>J13</b><br>Mer<br>05/08                | Sécurisation API : rôles admin / employé, validation stricte des DTO        | Gestion centralisée des erreurs et des exceptions HTTP              | Masquer le module Paie dans le dashboard si l'utilisateur n'est pas admin   | Confirmer que le module Paie est bien absent de l'application mobile   | Tests de sécurité basiques : accès non autorisé, données invalides, injection |
| <b>J14</b><br>Jeu<br>06/08                | Nettoyage du code, finalisation des derniers endpoints restants             | Nettoyage du code, finalisation des derniers endpoints restants     | Polish visuel, cohérence du design système sur tout le dashboard            | Polish visuel, cohérence du design système sur toute l'application     | Exécution officielle des 5 scénarios d'acceptation du cahier des charges      |
| <b>J15</b><br>Ven<br>07/08<br><b>DÉMO</b> | Correction des bugs Auth / Paie remontés par la QA                          | Correction des bugs Ingestion / Paie remontés par la QA             | Corrections finales avant la recette complète                               | Corrections finales avant la recette complète                          | Démo interne complète : toutes les fonctionnalités réunies                    |

## Semaine 4 — Tests, intégration hardware &amp; livraison

| Jour                                      | Backend 1<br>(Auth / Chantiers / Employés)                                    | Backend 2<br>(MQTT / Pointages / Paie)                                        | Frontend Web                                                    | Mobile                                                            | QA                                                                     |
|-------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|
| <b>J16</b><br>Lun<br>10/08                | Tests d'intégration avec simulateur MQTT (scénarios réalistes de terrain)     | Support aux tests d'intégration côté ingestion et paie                        | Correction des bugs remontés par la QA                          | Correction des bugs remontés par la QA                            | Piloter les tests d'intégration, consigner chaque anomalie             |
| <b>J17</b><br>Mar<br>11/08                | Tests de robustesse : coupures réseau simulées, gros volumes de pointages     | Optimisation des requêtes lentes identifiées lors des tests                   | Correction des bugs critiques restants                          | Correction des bugs critiques restants                            | Tests de charge et de robustesse ; validation des correctifs appliqués |
| <b>J18</b><br>Mer<br>12/08                | Rédaction du README du module Backend + schéma d'API à jour                   | Rédaction du README des modules Ingestion et Paie                             | Recette fonctionnelle complète : parcours admin de bout en bout | Recette fonctionnelle complète : parcours employé de bout en bout | Piloter la recette fonctionnelle bout en bout, consigner les résultats |
| <b>J19</b><br>Jeu<br>13/08                | Préparation de l'environnement de production (Docker, variables d'env)        | Vérification des migrations Prisma en conditions de production                | Corrections finales, gel des fonctionnalités (feature freeze)   | Corrections finales, gel des fonctionnalités (feature freeze)     | Dernière vérification des 5 critères d'acceptation avant livraison     |
| <b>J20</b><br>Ven<br>14/08<br><b>DÉMO</b> | Support de la démo finale, disponibilité pour les questions techniques du CEO | Support de la démo finale, disponibilité pour les questions techniques du CEO | Démonstration du dashboard admin au CEO                         | Démonstration de l'application mobile au CEO                      | Présentation du rapport de tests et de la recette finale               |